

```

In [2]: # CELL 1: Install YOLOv8 and verify our dataset
!pip install ultralytics -q # -q means "quiet", less messy output

import os
from pathlib import Path

# Where Kaggle puts the dataset
DATASET_ROOT = Path("/kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_D

# Let's verify every folder exists and count files
splits = {
    "Train": DATASET_ROOT / "VisDrone2019-DET-train",
    "Val":    DATASET_ROOT / "VisDrone2019-DET-val",
    "Test":   DATASET_ROOT / "VisDrone2019-DET-test-dev",
}

print("=" * 50)
print("DATASET VERIFICATION REPORT")
print("=" * 50)

all_good = True
for split_name, split_path in splits.items():
    img_path = split_path / "images"
    lbl_path = split_path / "labels"

    # Count files
    images = sorted([f.stem for f in img_path.glob("*.jpg")] +
                    [f.stem for f in img_path.glob("*.png")])

    if lbl_path.exists():
        labels = sorted([f.stem for f in lbl_path.glob("*.txt")])
        matched = len(set(images) & set(labels))
        only_img = len(set(images) - set(labels))
        only_lbl = len(set(labels) - set(images))

        print(f"\n{split_name}:")
        print(f"  Images:  {len(images)}")
        print(f"  Labels:  {len(labels)}")
        print(f"  Matched pairs: {matched} ✓")
        if only_img > 0:
            print(f"    △ Images WITHOUT labels: {only_img}")
            all_good = False
        if only_lbl > 0:
            print(f"    △ Labels WITHOUT images: {only_lbl}")
            all_good = False
    else:
        print(f"\n{split_name}: No labels folder (test-challenge is normal)")

print("\n" + "=" * 50)
if all_good:
    print("✅ Dataset is CLEAN – all images have matching labels!")
else:
    print("△ Some mismatches found (see above)")
print("=" * 50)

```

1.2/1.2 MB 44.7 MB/s eta 0:00:00

=====

DATASET VERIFICATION REPORT

=====

Train:

Images: 6471
 Labels: 6471
 Matched pairs: 6471 ✓

Val:

Images: 548
 Labels: 548
 Matched pairs: 548 ✓

Test:

Images: 1610
 Labels: 1610
 Matched pairs: 1610 ✓

=====

✓ Dataset is CLEAN – all images have matching labels!

=====

```
In [4]: # CELL 2: Visualize sample images with their bounding boxes (Task 1)
import cv2
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np
import random

# Class names and colors (one color per class for clarity)
CLASS_NAMES = ['pedestrian', 'people', 'bicycle', 'car', 'van',
               'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']
COLORS = plt.get_cmap('tab10', 10)

def show_sample_images(split_path, num_samples=4, title="Sample"):
    img_dir = split_path / "images"
    lbl_dir = split_path / "labels"

    all_imgs = list(img_dir.glob("*.jpg")) + list(img_dir.glob("*.png"))
    samples = random.sample(all_imgs, min(num_samples, len(all_imgs)))

    fig, axes = plt.subplots(1, num_samples, figsize=(20, 5))
    fig.suptitle(title, fontsize=14, fontweight='bold')

    for ax, img_path in zip(axes, samples):
        # Load image
        img = cv2.imread(str(img_path))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        h, w = img.shape[:2]

        ax.imshow(img)
        ax.set_title(img_path.name[:20], fontsize=8)
        ax.axis('off')
```

```

# Load corresponding label file
lbl_path = lbl_dir / (img_path.stem + ".txt")
if lbl_path.exists():
    with open(lbl_path) as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) < 5:
                continue
            cls = int(parts[0])
            cx, cy, bw, bh = map(float, parts[1:5])

            # Convert from YOLO format (0-1 ratios) to pixel coords
            x1 = (cx - bw/2) * w
            y1 = (cy - bh/2) * h

            color = COLORS[cls][:3]
            rect = patches.Rectangle(
                (x1, y1), bw*w, bh*h,
                linewidth=1, edgecolor=color, facecolor='none'
            )
            ax.add_patch(rect)
            ax.text(x1, y1-2, CLASS_NAMES[cls],
                    fontsize=5, color=color)

plt.tight_layout()
plt.savefig(f"/kaggle/working/{title.replace(' ', '_')}.png",
            dpi=150, bbox_inches='tight')

plt.show()
print(f"Saved to /kaggle/working/")

# Run for train set
random.seed(42) # So results are reproducible
show_sample_images(splits["Train"], title="Train Set Samples")
show_sample_images(splits["Val"], title="Validation Set Samples")

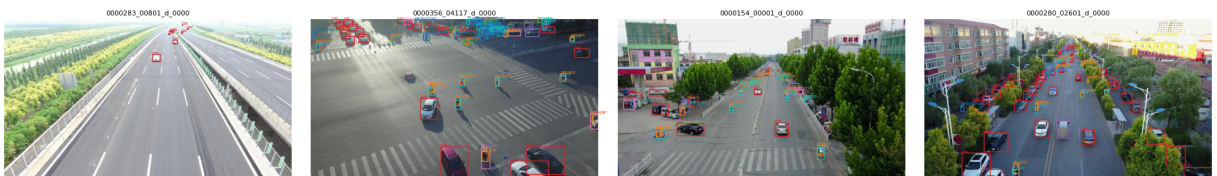
```

Train Set Samples



Saved to /kaggle/working/

Validation Set Samples



Saved to /kaggle/working/

In [6]: # CELL 3: Write the dataset config file that YOLOv8 needs
import yaml

```
# We write the YAML pointing to Kaggle's input paths
config = {
    'path': '/kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset',
    'train': 'VisDrone2019-DET-train/images',
    'val': 'VisDrone2019-DET-val/images',
    'test': 'VisDrone2019-DET-test-dev/images',
    'nc': 10,
    'names': CLASS_NAMES
}

config_path = '/kaggle/working/visdrone.yaml'
with open(config_path, 'w') as f:
    yaml.dump(config, f, default_flow_style=False)

print("✅ visdrone.yaml written to:", config_path)
print("\nContents:")
with open(config_path) as f:
    print(f.read())
```

✅ visdrone.yaml written to: /kaggle/working/visdrone.yaml

Contents:

names:

- pedestrian
- people
- bicycle
- car
- van
- truck
- tricycle
- awning-tricycle
- bus
- motor

nc: 10

path: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset

test: VisDrone2019-DET-test-dev/images

train: VisDrone2019-DET-train/images

val: VisDrone2019-DET-val/images

```
In [7]: # CELL 4: TRAIN YOLOv8s on VisDrone
# YOLOv8s = "small" model – good balance of speed and accuracy
from ultralytics import YOLO

# Load a pretrained YOLOv8s model (pretrained on COCO dataset)
# We "fine-tune" it on VisDrone – much faster than training from scratch
model = YOLO('yolov8s.pt')

# Start training!
results = model.train(
    data='/kaggle/working/visdrone.yaml',
    epochs=50,          # 60 passes through the whole dataset
    imgsz=800,          # image size in pixels
    batch=16,           # process 16 images at once (fits P100 memory)
    workers=2,
```



```
device=0,          # use GPU (device 0)
project='/kaggle/working/runs',
name='visdrone_train',

# Augmentation (makes model more robust – built into ultralytics)
hsv_h=0.015,      # slight color shifts
hsv_s=0.7,
hsv_v=0.4,
flipud=0.0,       # drone images can be flipped
fliplr=0.5,
mosaic=1.0,       # mixes 4 images (great for small objects)
close_mosaic=10,  # disable mosaic for last 10 epochs for cleaner final learning

# Save best model
save=True,
save_period=5,    # save checkpoint every 5 epochs

verbose=True      # show progress
)

print("\n✅ TRAINING COMPLETE!")
print(f"Best model saved at: /kaggle/working/runs/visdrone_train/weights/best.pt")
```

Creating new Ultralytics Settings v0.0.6 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com/quickstart/#ultralytics-settings>.
 Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8s.pt> to 'yolov8s.pt': 100% ————— 21.5MB 467.9MB/s 0.0s

Ultralytics 8.4.51  Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=RandAugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, cls_pw=0.0, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=/kaggle/working/visdrone.yaml, degrees=0.0, deterministic=True, device=0, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None, end2end=None, epochs=50, erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=800, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8s.pt, momentum=0.937, mosaic=1.0, multi_scale=0.0, name=visdrone_train, nbs=64, nms=False, opset=None, optimize=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=True, profile=False, project=/kaggle/working/runs, rect=False, resume=False, retina_masks=False, rle=1.0, save=True, save_conf=False, save_crop=False, save_dir=/kaggle/working/runs/visdrone_train, save_frames=False, save_json=False, save_period=5, save_txt=False, scale=0.5, seed=0, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=detect, time=None, tracker=botsort.yaml, translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=2, workspace=None

Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/Arial.ttf': 100% ————— 755.1KB 116.2MB/s 0.0s

Overriding model.yaml nc=80 with nc=10

	from	n	params	module	a
arguments					
0	-1	1	928	ultralytics.nn.modules.conv.Conv	
[3, 32, 3, 2]					
1	-1	1	18560	ultralytics.nn.modules.conv.Conv	
[32, 64, 3, 2]					
2	-1	1	29056	ultralytics.nn.modules.block.C2f	
[64, 64, 1, True]					
3	-1	1	73984	ultralytics.nn.modules.conv.Conv	
[64, 128, 3, 2]					
4	-1	2	197632	ultralytics.nn.modules.block.C2f	
[128, 128, 2, True]					
5	-1	1	295424	ultralytics.nn.modules.conv.Conv	
[128, 256, 3, 2]					
6	-1	2	788480	ultralytics.nn.modules.block.C2f	
[256, 256, 2, True]					
7	-1	1	1180672	ultralytics.nn.modules.conv.Conv	
[256, 512, 3, 2]					
8	-1	1	1838080	ultralytics.nn.modules.block.C2f	
[512, 512, 1, True]					
9	-1	1	656896	ultralytics.nn.modules.block.SPPF	
[512, 512, 5]					
10	-1	1	0	torch.nn.modules.upsampling.Upsample	
[None, 2, 'nearest']					

```

11          [-1, 6] 1          0 ultralytics.nn.modules.conv.Concat
[1]
12          -1 1      591360 ultralytics.nn.modules.block.C2f
[768, 256, 1]
13          -1 1          0 torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']
14          [-1, 4] 1          0 ultralytics.nn.modules.conv.Concat
[1]
15          -1 1      148224 ultralytics.nn.modules.block.C2f
[384, 128, 1]
16          -1 1      147712 ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
17          [-1, 12] 1          0 ultralytics.nn.modules.conv.Concat
[1]
18          -1 1      493056 ultralytics.nn.modules.block.C2f
[384, 256, 1]
19          -1 1      590336 ultralytics.nn.modules.conv.Conv
[256, 256, 3, 2]
20          [-1, 9] 1          0 ultralytics.nn.modules.conv.Concat
[1]
21          -1 1      1969152 ultralytics.nn.modules.block.C2f
[768, 512, 1]
22          [15, 18, 21] 1      2119918 ultralytics.nn.modules.head.Detect
[10, 16, None, [128, 256, 512]]
Model summary: 130 layers, 11,139,470 parameters, 11,139,454 gradients, 28.7 GFLOPs

```

Transferred 349/355 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to 'yolo26n.pt': 100% ————— 5.3MB 289.5MB/s 0.0s

AMP: checks passed 

train: Fast image access  (ping: 0.9±0.3 ms, read: 26.5±7.4 MB/s, size: 260.7 KB)

train: Scanning /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-train/labels... 6471 images, 0 backgrounds, 0 corrupt: 100% —————
6471/6471 183.4it/s 35.3s<0.0s

train: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-train/images/0000137_02220_d_0000163.jpg: 1 duplicate labels removed

train: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-train/images/0000140_00118_d_0000002.jpg: 1 duplicate labels removed

train: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-train/images/9999945_00000_d_0000114.jpg: 1 duplicate labels removed

train: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-train/images/9999987_00000_d_0000049.jpg: 1 duplicate labels removed

WARNING  **train:** Cache directory /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-train is not writable, cache not saved.

augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

val: Fast image access  (ping: 0.0±0.0 ms, read: 21.1±6.2 MB/s, size: 142.1 KB)

val: Scanning /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-val/labels... 548 images, 0 backgrounds, 0 corrupt: 100% —————
548/548 190.1it/s 2.9s0.1s

WARNING  **val:** Cache directory /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-val is not writable, cache not saved.

optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and dete

mining best 'optimizer', 'lr0' and 'momentum' automatically...

optimizer: AdamW(lr=0.000714, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)

Plotting labels to /kaggle/working/runs/visdrone_train/labels.jpg...

Image sizes 800 train, 800 val

Using 2 dataloader workers

Logging results to /kaggle/working/runs/visdrone_train

Starting training for 50 epochs...

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
1/50	13.2G	1.581	1.692	1.001	655	800: 100%
	405/405	2.1it/s	3:16<0.5s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%	18/18		2.4it/s	7.4s0.4s		
	all	548	38759	0.362	0.3	0.251

0.144

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
2/50	13.4G	1.5	1.195	0.9608	477	800: 100%
	405/405	2.1it/s	3:17<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%	18/18		2.8it/s	6.5s0.4s		
	all	548	38759	0.408	0.319	0.275

0.154

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
3/50	11.9G	1.498	1.169	0.9552	677	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%	18/18		2.8it/s	6.5s0.4s		
	all	548	38759	0.422	0.336	0.3

0.17

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
4/50	11.7G	1.469	1.127	0.949	672	800: 100%
	405/405	2.1it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%	18/18		2.7it/s	6.6s0.4s		
	all	548	38759	0.433	0.356	0.321

0.183

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
5/50	9.67G	1.45	1.1	0.9444	846	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%	18/18		2.8it/s	6.5s0.4s		
	all	548	38759	0.443	0.362	0.335

0.193

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
6/50	13G	1.446	1.077	0.9394	323	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%	18/18		2.8it/s	6.4s0.4s		
	all	548	38759	0.455	0.363	0.336

0.196

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
7/50	10.7G	1.428	1.048	0.9348	598	800: 100%	
	405/405	2.0it/s	3:18<0.4s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%							
	all	18/18	2.8it/s	6.4s0.4s			
		548	38759	0.474	0.371	0.351	
0.206							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
8/50	14.1G	1.419	1.033	0.9299	500	800: 100%	
	405/405	2.0it/s	3:19<0.4s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%							
	all	18/18	2.8it/s	6.3s0.4s			
		548	38759	0.472	0.368	0.348	
0.204							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
9/50	13.1G	1.401	1.017	0.9283	669	800: 100%	
	405/405	2.0it/s	3:18<0.4s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%							
	all	18/18	2.8it/s	6.5s0.4s			
		548	38759	0.487	0.371	0.36	
0.21							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
10/50	13.3G	1.407	1.016	0.9277	668	800: 100%	
	405/405	2.0it/s	3:18<0.4s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%							
	all	18/18	2.8it/s	6.4s0.4s			
		548	38759	0.477	0.375	0.362	
0.208							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
11/50	12.1G	1.396	0.997	0.9244	656	800: 100%	
	405/405	2.0it/s	3:18<0.4s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%							
	all	18/18	2.8it/s	6.4s0.4s			
		548	38759	0.504	0.376	0.366	
0.213							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
12/50	14.3G	1.392	0.9839	0.9203	920	800: 100%	
	405/405	2.0it/s	3:19<0.4s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%							
	all	18/18	2.8it/s	6.4s0.4s			
		548	38759	0.497	0.394	0.381	
0.222							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
13/50	10.8G	1.381	0.979	0.9212	275	800: 100%	
	405/405	2.0it/s	3:18<0.3s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%							
	all	18/18	2.8it/s	6.5s0.4s			
		548	38759	0.51	0.399	0.386	
0.224							

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
14/50	12.9G	1.367	0.9661	0.9176	623	800: 100%
	405/405	2.1it/s	3:17<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.3s0.4s			
	all	548	38759	0.522	0.396	0.39
0.227						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
15/50	11.1G	1.365	0.9533	0.9144	551	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.3s0.4s			
	all	548	38759	0.486	0.403	0.388
0.226						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
16/50	9.54G	1.36	0.9479	0.9127	899	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.8it/s	6.4s0.4s			
	all	548	38759	0.507	0.393	0.387
0.228						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
17/50	11.2G	1.358	0.9447	0.9117	584	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.2s0.4s			
	all	548	38759	0.505	0.407	0.395
0.232						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
18/50	12.2G	1.356	0.9391	0.9141	413	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.8it/s	6.3s0.4s			
	all	548	38759	0.527	0.411	0.405
0.237						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
19/50	10.4G	1.343	0.929	0.9088	317	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.3s0.4s			
	all	548	38759	0.527	0.413	0.409
0.241						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
20/50	11.5G	1.342	0.917	0.9088	858	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.3s0.4s			
	all	548	38759	0.524	0.415	0.405
0.237						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
21/50	11.3G	1.338	0.9137	0.9083	493	800: 100%
	405/405	2.0it/s	3:19<0.3s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.8it/s	6.3s0.4s		
		548	38759	0.53	0.416	0.407
0.24						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
22/50	11.3G	1.344	0.9138	0.906	731	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.526	0.411	0.407
0.24						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
23/50	13.7G	1.324	0.899	0.9044	482	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.529	0.422	0.418
0.246						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
24/50	12.7G	1.318	0.8891	0.9041	680	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.534	0.415	0.414
0.244						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
25/50	13.6G	1.312	0.8864	0.9023	374	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.8it/s	6.4s0.4s		
		548	38759	0.548	0.417	0.421
0.249						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
26/50	12.5G	1.314	0.8826	0.9006	539	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.8it/s	6.3s0.4s		
		548	38759	0.53	0.425	0.418
0.246						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
27/50	12.9G	1.307	0.8748	0.9011	678	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.541	0.424	0.42
0.249						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
28/50	11.3G	1.313	0.8697	0.8981	740	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.3s0.4s			
	all	548	38759	0.551	0.423	0.421
0.249						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
29/50	14.2G	1.303	0.8678	0.8958	426	800: 100%
	405/405	2.0it/s	3:18<0.3s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.2s0.4s			
	all	548	38759	0.536	0.425	0.423
0.248						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
30/50	12G	1.296	0.8575	0.8974	620	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.8it/s	6.3s0.4s			
	all	548	38759	0.54	0.427	0.426
0.251						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
31/50	12.8G	1.3	0.8569	0.8955	670	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.2s0.4s			
	all	548	38759	0.545	0.431	0.428
0.253						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
32/50	11.4G	1.296	0.8526	0.894	407	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.8it/s	6.3s0.4s			
	all	548	38759	0.553	0.426	0.431
0.254						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
33/50	12.3G	1.29	0.8454	0.8943	711	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.2s0.4s			
	all	548	38759	0.552	0.432	0.429
0.255						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
34/50	12.4G	1.279	0.834	0.8939	910	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	18/18	2.9it/s	6.3s0.4s			
	all	548	38759	0.55	0.438	0.433
0.257						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
35/50	13G	1.277	0.8289	0.8916	454	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.2s0.4s		
		548	38759	0.544	0.437	0.435
0.258						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
36/50	13.7G	1.277	0.8285	0.8897	784	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.556	0.432	0.431
0.255						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
37/50	14G	1.272	0.8246	0.8914	517	800: 100%
	405/405	2.1it/s	3:17<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.54	0.437	0.432
0.256						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
38/50	11.4G	1.267	0.8165	0.8888	860	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.2s0.4s		
		548	38759	0.543	0.445	0.437
0.258						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
39/50	11.1G	1.264	0.8109	0.887	786	800: 100%
	405/405	2.0it/s	3:19<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.558	0.438	0.436
0.258						

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
40/50	13.6G	1.262	0.8055	0.8877	754	800: 100%
	405/405	2.0it/s	3:18<0.4s			
	Class	Images	Instances	Box(P	R	mAP50 mAP50
-95): 100%						
	all	18/18	2.9it/s	6.3s0.4s		
		548	38759	0.55	0.443	0.437
0.258						

Closing dataloader mosaic

alumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
41/50	10.6G	1.233	0.7831	0.8926	230	800: 100%
	405/405	2.2it/s	3:06<0.3s			

	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.545	0.444	0.434	
0.256							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
42/50	12.4G	1.22	0.7697	0.8897	282	800: 100%	
	405/405	2.2it/s	3:05<0.3s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.541	0.442	0.435	
0.256							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
43/50	11.2G	1.217	0.7647	0.889	269	800: 100%	
	405/405	2.2it/s	3:04<0.3s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.556	0.441	0.435	
0.257							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
44/50	11.4G	1.212	0.7571	0.8876	545	800: 100%	
	405/405	2.2it/s	3:05<0.4s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.549	0.438	0.434	
0.258							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
45/50	11.9G	1.208	0.7505	0.8858	384	800: 100%	
	405/405	2.2it/s	3:05<0.3s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.552	0.433	0.432	
0.256							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
46/50	13.7G	1.206	0.7467	0.8862	231	800: 100%	
	405/405	2.2it/s	3:05<0.3s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.551	0.442	0.436	
0.258							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
47/50	11G	1.198	0.7393	0.8844	335	800: 100%	
	405/405	2.2it/s	3:05<0.3s				
	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.552	0.442	0.437	
0.258							
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
48/50	11.3G	1.199	0.7395	0.884	323	800: 100%	
	405/405	2.2it/s	3:05<0.4s				

	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.561	0.441	0.438	
0.258							

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
49/50	13.3G	1.192	0.7306	0.8818	269	800: 100%
405/405 2.2it/s 3:05<0.3s						

	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.2s0.4s	0.553	0.444	0.437	
0.258							

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
50/50	14.3G	1.193	0.7311	0.8817	375	800: 100%
405/405 2.2it/s 3:05<0.3s						

	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 2.9it/s	6.1s0.4s	0.559	0.44	0.438	
0.258							

50 epochs completed in 2.819 hours.

Optimizer stripped from /kaggle/working/runs/visdrone_train/weights/last.pt, 22.5MB

Optimizer stripped from /kaggle/working/runs/visdrone_train/weights/best.pt, 22.5MB

Validating /kaggle/working/runs/visdrone_train/weights/best.pt...

Ultralytics 8.4.51 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 73 layers, 11,129,454 parameters, 0 gradients, 28.5 GFLOPs

	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%	all	18/18 1.8it/s	9.9s0.3s	0.55	0.443	0.436	
0.258							
0.227	pedestrian	520	8844	0.557	0.503	0.5	
0.146	people	482	5125	0.585	0.364	0.377	
0.0856	bicycle	364	1287	0.315	0.236	0.192	0.
0.57	car	515	14064	0.751	0.814	0.822	
0.334	van	421	1975	0.561	0.478	0.477	
0.278	truck	266	750	0.555	0.405	0.413	
0.185	tricycle	337	1045	0.45	0.364	0.326	
0.101	awning-tricycle	220	532	0.337	0.216	0.162	
0.432	bus	131	251	0.783	0.558	0.585	
0.226	motor	485	4886	0.603	0.496	0.509	

Speed: 0.2ms preprocess, 3.8ms inference, 0.0ms loss, 2.0ms postprocess per image

Results saved to /kaggle/working/runs/visdrone_train

✅ TRAINING COMPLETE!

Best model saved at: /kaggle/working/runs/visdrone_train/weights/best.pt

```
In [8]: # CELL 5: Look at training results
# Auto-detect whichever run has the better mAP and present the results

import pandas as pd
from pathlib import Path
from IPython.display import Image as IImage, display

runs_root = Path('/kaggle/working/runs')

# Find Cell 4 training runs only
candidate_runs = sorted(
    [p for p in runs_root.glob('visdrone_train*') if p.is_dir()],
    key=lambda p: p.stat().st_mtime,
    reverse=True
)

if not candidate_runs:
    raise FileNotFoundError("No visdrone_train run folder found under /kaggle/worki

# Use the latest Cell 4 run
results_dir = candidate_runs[0]
metrics_file = results_dir / 'results.csv'

print(f"Using training run: {results_dir}")

if not metrics_file.exists():
    raise FileNotFoundError(f"results.csv not found in: {results_dir}")

df = pd.read_csv(metrics_file)
df.columns = df.columns.str.strip()

# Metrics columns from Ultralytics
map50_col = 'metrics/mAP50(B)'
map5095_col = 'metrics/mAP50-95(B)'

if map50_col not in df.columns:
    raise KeyError(f"Column not found: {map50_col}. Available columns: {list(df.col

best_map50 = df[map50_col].max()
best_map50_epoch = df[map50_col].idxmax() + 1

print("\n📊 TRAINING SUMMARY")
print(f"Total completed epochs: {len(df)}")
print(f"Best mAP@50: {best_map50:.4f} (epoch {best_map50_epoch})")

if map5095_col in df.columns:
    best_map5095 = df[map5095_col].max()
    best_map5095_epoch = df[map5095_col].idxmax() + 1
    print(f"Best mAP@50-95: {best_map5095:.4f} (epoch {best_map5095_epoch})

# Show final epoch values too
last = df.iloc[-1]
print("\nFinal epoch metrics:")
```

```

print(f"mAP@50: {last[map50_col]:.4f}")

if map5095_col in df.columns:
    print(f"mAP@50-95: {last[map5095_col]:.4f}")

# Display Ultralytics results plot
results_img = results_dir / 'results.png'

if results_img.exists():
    print("\nTraining curve:")
    display(IPImage(str(results_img), width=900))
else:
    print("\nresults.png not found yet.")

# Define model path for all later cells
best_model_path = results_dir / 'weights' / 'best.pt'
last_model_path = results_dir / 'weights' / 'last.pt'

if best_model_path.exists():
    MODEL_PATH = str(best_model_path)
elif last_model_path.exists():
    MODEL_PATH = str(last_model_path)
    print("⚠ best.pt not found, using last.pt instead.")
else:
    raise FileNotFoundError(f"No best.pt or last.pt found in: {results_dir / 'weights'}")

print(f"\n✅ Using model: {MODEL_PATH}")

```

Using training run: /kaggle/working/runs/visdrone_train

TRAINING SUMMARY

Total completed epochs: 50

Best mAP@50: 0.4380 (epoch 50)

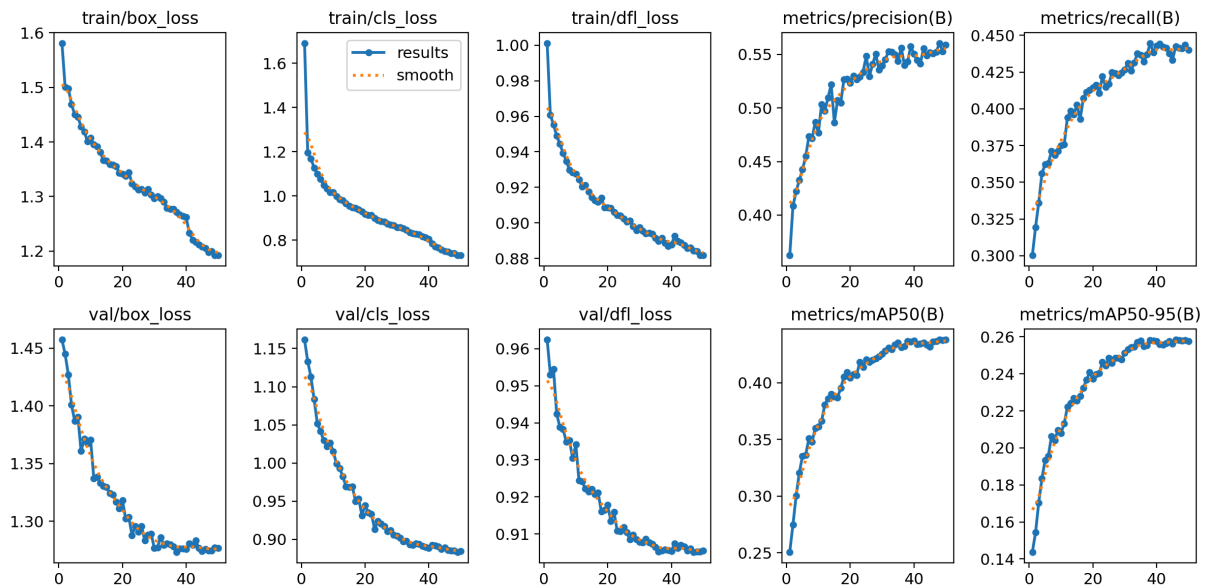
Best mAP@50-95: 0.2584 (epoch 46)

Final epoch metrics:

mAP@50: 0.4380

mAP@50-95: 0.2576

Training curve:



✓ Using model: /kaggle/working/runs/visdrone_train/weights/best.pt

```
In [11]: # CELL 6: Detection + Human Counting (The core deliverable)
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path

# Load the best trained model
model = YOLO(MODEL_PATH)

# Which classes we care about for this task
HUMAN_CLASSES = {0: 'pedestrian', 1: 'people'} # count these
CAR_CLASSES    = {3: 'car', 4: 'van'}           # detect these

def detect_and_count(image_path, conf_threshold=0.25):
    """
    Runs detection on one image.
    Returns: annotated image (numpy array) + counts dict
    """
    img = cv2.imread(str(image_path))
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    h, w = img.shape[:2]

    # Run inference
    results = model(img_rgb, conf=conf_threshold, verbose=False)[0]

    # Count humans and cars
    human_count = 0
    car_count = 0

    # Draw boxes
    annotated = img_rgb.copy()

    for box in results boxes:
        cls_id = int(box.cls[0])
        conf = float(box.conf[0])
```

```

x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())

if cls_id in HUMAN_CLASSES:
    human_count += 1
    color = (255, 80, 80) # red for humans
    label = f"{HUMAN_CLASSES[cls_id]} {conf:.2f}"
elif cls_id in CAR_CLASSES:
    car_count += 1
    color = (80, 180, 255) # blue for cars
    label = f"car {conf:.2f}"
else:
    continue # skip other classes like bicycle, truck etc.

# Draw bounding box
cv2.rectangle(annotated, (x1, y1), (x2, y2), color, 2)

# Draw Label background
(tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.4, 1)
cv2.rectangle(annotated, (x1, y1-th-4), (x1+tw, y1), color, -1)
cv2.putText(annotated, label, (x1, y1-2),
            cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255,255,255), 1)

# Draw the count banner at the top of the image
banner_h = 40
banner = np.zeros((banner_h, w, 3), dtype=np.uint8)
banner[:] = (30, 30, 30) # dark background
cv2.putText(banner, f"HUMANS: {human_count}", (10, 28),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 80, 80), 2)
cv2.putText(banner, f"CARS: {car_count}", (250, 28),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (80, 180, 255), 2)
cv2.putText(banner, f"conf > {conf_threshold}", (w-200, 28),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (200, 200, 200), 1)

# Stack banner on top of image
final = np.vstack([banner, annotated])

return final, {'humans': human_count, 'cars': car_count}

# --- Run on 6 test images and display ---
test_img_dir = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_D
# test_images = sorted(test_img_dir.glob("*.jpg"))[:6] # first 6 images
imgs = list(test_img_dir.glob("*.jpg")) + \
        list(test_img_dir.glob("*.JPG")) + \
        list(test_img_dir.glob("*.png"))
test_images = sorted(imgs)[:6]
print(f"Found {len(imgs)} images, using first 6")

fig, axes = plt.subplots(2, 3, figsize=(22, 12))
fig.suptitle("Human & Car Detection Results", fontsize=16, fontweight='bold')
axes = axes.flatten()

for ax, img_path in zip(axes, test_images):
    result_img, counts = detect_and_count(img_path)
    ax.imshow(result_img)
    ax.set_title(f"Humans: {counts['humans']} | Cars: {counts['cars']}", fontsize

```



```
ax.axis('off')

plt.tight_layout()
plt.savefig('/kaggle/working/detection_results.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: /kaggle/working/detection_results.png")
```

Found 1610 images, using first 6

Human & Car Detection Results



✅ Saved: /kaggle/working/detection_results.png

```
In [12]: # CELL 6-B: SAHI Sliced Inference for better tiny-object detection.
#
# The core idea: instead of shrinking the whole image to 640px (which makes
# tiny people invisible), we cut the image into overlapping 640x640 tiles,
# run detection on each tile at full detail, then merge all the boxes back.
# This is the standard technique for drone/satellite imagery.
```

```
!pip install sahi -q
```

```
from sahi import AutoDetectionModel
from sahi.predict import get_sliced_prediction
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import cv2
import numpy as np
from pathlib import Path
```

```
# Point SAHI to whichever model finished training (resumed if you ran 4-B,
# otherwise the original best)
```

```
print(f"Using model: {MODEL_PATH}")
```

```
# Load the model through SAHI's wrapper (it internally uses ultralytics)
sahi_model = AutoDetectionModel.from_pretrained(
    model_type='ultralytics',
    model_path=MODEL_PATH,
    confidence_threshold=0.25,
    device='cuda:0')
```

```

)

# Class definitions – same logic as before
HUMAN_CLASSES = {'pedestrian', 'people'}
CAR_CLASSES    = {'car', 'van'}

# Your 10 class names in order – must match the yaml exactly
CLASS_NAMES = ['pedestrian', 'people', 'bicycle', 'car', 'van',
               'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']

def detect_with_sahi(image_path):
    """
    Runs sliced detection on a single image.
    slice_height/width: size of each tile (640 matches YOLO's training size)
    overlap_height/width_ratio: 20% overlap prevents missing objects at tile edges
    """
    result = get_sliced_prediction(
        str(image_path),
        sahi_model,
        slice_height=800,
        slice_width=800,
        overlap_height_ratio=0.2,
        overlap_width_ratio=0.2,
        verbose=0
    )

    # Load image for drawing
    img = cv2.imread(str(image_path))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    h, w = img.shape[:2]

    human_count = 0
    car_count    = 0

    for obj in result.object_prediction_list:
        class_name = obj.category.name
        bbox = obj.bbox # SAHI bbox object
        x1, y1, x2, y2 = int(bbox.minx), int(bbox.miny), int(bbox.maxx), int(bbox.m

        if class_name in HUMAN_CLASSES:
            human_count += 1
            color = (255, 80, 80)
            label = f"{class_name} {obj.score.value:.2f}"
        elif class_name in CAR_CLASSES:
            car_count += 1
            color = (80, 180, 255)
            label = f"car {obj.score.value:.2f}"
        else:
            continue # skip non-target classes

        cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
        (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.4, 1)
        cv2.rectangle(img, (x1, y1-th-4), (x1+tw, y1), color, -1)
        cv2.putText(img, label, (x1, y1-2),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255,255,255), 1)

```

```

# Count banner
banner = np.zeros((45, w, 3), dtype=np.uint8)
banner[:] = (25, 25, 25)
cv2.putText(banner, f"[SAHI] HUMANS: {human_count}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 80, 80), 2)
cv2.putText(banner, f"CARS: {car_count}", (350, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (80, 180, 255), 2)

return np.vstack([banner, img]), {'humans': human_count, 'cars': car_count}

# --- Compare standard detection vs SAHI on the same 3 images ---
# This side-by-side comparison is great to include in your report
test_img_dir = Path(
    '/kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2
)
test_images = sorted(test_img_dir.glob("*.jpg"))[:3]

# We need the regular detect_and_count function from Cell 6 as well
# (make sure Cell 6 has been run before this cell)

fig, axes = plt.subplots(3, 2, figsize=(22, 18))
fig.suptitle("Standard Detection (left) vs SAHI Sliced Detection (right)\n"
            "SAHI catches more tiny people by processing the image in detail tiles
            fontsize=13, fontweight='bold')

for row, img_path in enumerate(test_images):
    # Left column: regular detection from Cell 6
    regular_img, regular_counts = detect_and_count(img_path)
    axes[row, 0].imshow(regular_img)
    axes[row, 0].set_title(
        f"Standard - Humans: {regular_counts['humans']} Cars: {regular_counts['car
        fontsize=10
    )
    axes[row, 0].axis('off')

    # Right column: SAHI detection
    sahi_img, sahi_counts = detect_with_sahi(img_path)
    axes[row, 1].imshow(sahi_img)
    axes[row, 1].set_title(
        f"SAHI - Humans: {sahi_counts['humans']} Cars: {sahi_counts['cars']}",
        fontsize=10
    )
    axes[row, 1].axis('off')

plt.tight_layout()
plt.savefig('/kaggle/working/sahi_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: /kaggle/working/sahi_comparison.png")
print("\nIn your demo, point out how SAHI detects more small humans in crowded area

```


111.7/111.7 kB 8.5 MB/s eta 0:00:00

63.0/63.0 MB 30.0 MB/s eta 0:00:00:00:01

00:01

115.9/115.9 kB 7.0 MB/s eta 0:00:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

dopamine-rl 4.1.2 requires gym<=0.25.2, but you have gym 0.26.2 which is incompatible.

Using model: /kaggle/working/runs/visdrone_train/weights/best.pt



✓ Saved: /kaggle/working/sahi_comparison.png

In your demo, point out how SAHI detects more small humans in crowded areas.

```
In [14]: # RESET CELL – updated fix, use dicts not sets
HUMAN_CLASSES = {0: 'pedestrian', 1: 'people'} # dict: key=class ID, value=name
CAR_CLASSES    = {3: 'car', 4: 'van'}           # dict: key=class ID, value=name
```

```
In [15]: # CELL 7: Run detection on ALL test images and save summary counts

from pathlib import Path
from tqdm import tqdm
import pandas as pd
```

```

import numpy as np
from ultralytics import YOLO

model_best = YOLO(MODEL_PATH)

# VisDrone class IDs
HUMAN_CLASSES = {0, 1} # pedestrian, people
CAR_CLASSES = {3, 4} # car only
TARGET_CLASSES = HUMAN_CLASSES | CAR_CLASSES

# Automatically find test-dev/images folder
root = Path('/kaggle/input')

candidate_dirs = []
for d in root.rglob('images'):
    imgs = list(d.glob('*.jpg')) + list(d.glob('*.JPG')) + list(d.glob('*.png'))
    if len(imgs) > 0 and 'VisDrone2019-DET-test-dev' in str(d):
        candidate_dirs.append((d, len(imgs)))

if not candidate_dirs:
    raise FileNotFoundError("Could not find VisDrone2019-DET-test-dev/images under

test_img_dir = max(candidate_dirs, key=lambda x: x[1])[0]
print("Using test image folder:", test_img_dir)

all_test_images = sorted(
    list(test_img_dir.glob("*.jpg")) +
    list(test_img_dir.glob("*.JPG")) +
    list(test_img_dir.glob("*.jpeg")) +
    list(test_img_dir.glob("*.png"))
)

if len(all_test_images) == 0:
    raise FileNotFoundError(f"No test images found in: {test_img_dir}")

print(f"Running detection on {len(all_test_images)} test images...")

def count_only(image_path, conf_threshold=0.25, imgsz=800):
    """
    Fast counting function.
    Does not draw images, only counts detected humans and cars.
    """
    result = model_best.predict(
        source=str(image_path),
        conf=conf_threshold,
        imgsz=imgsz,
        classes=list(TARGET_CLASSES),
        verbose=False
    )[0]

    if result.bboxes is None or len(result.bboxes) == 0:
        return {
            'humans': 0,
            'cars': 0,
            'total_target_objects': 0

```

```

    }

    cls_ids = result.bboxes.cls.cpu().numpy().astype(int)

    human_count = int(np.isin(cls_ids, list(HUMAN_CLASSES)).sum())
    car_count = int(np.isin(cls_ids, list(CAR_CLASSES)).sum())

    return {
        'humans': human_count,
        'cars': car_count,
        'total_target_objects': human_count + car_count
    }

results_log = []

for img_path in tqdm(all_test_images):
    counts = count_only(img_path, conf_threshold=0.25, imsz=800)

    results_log.append({
        'image': img_path.name,
        'humans': counts['humans'],
        'cars': counts['cars'],
        'total_target_objects': counts['total_target_objects']
    })

df_results = pd.DataFrame(results_log)
csv_path = '/kaggle/working/detection_log.csv'
df_results.to_csv(csv_path, index=False)

total_humans = int(df_results['humans'].sum())
total_cars = int(df_results['cars'].sum())

print(f"\n{'='*45}")
print("DETECTION COUNT SUMMARY")
print(f"{'='*45}")
print(f"Images processed: {len(df_results)}")
print(f"Total humans detected: {total_humans}")
print(f"Total cars detected: {total_cars}")
print(f"Avg humans per image: {df_results['humans'].mean():.2f}")
print(f"Avg cars per image: {df_results['cars'].mean():.2f}")
print(f"Images with no humans: {(df_results['humans'] == 0).sum()}")
print(f"Images with no cars: {(df_results['cars'] == 0).sum()}")
print(f"Saved CSV: {csv_path}")
print(f"{'='*45}")

print("\nTop 10 images by human count:")
print(df_results.nlargest(10, 'humans')[['image', 'humans', 'cars']].to_string(index=False))

print("\nTop 10 images by car count:")
print(df_results.nlargest(10, 'cars')[['image', 'humans', 'cars']].to_string(index=False))

```

Using test image folder: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone2019-DET-test-dev/images
Running detection on 1610 test images...

100%|██████████| 1610/1610 [00:55<00:00, 29.13it/s]

```
=====
DETECTION COUNT SUMMARY
=====
Images processed:      1610
Total humans detected: 13582
Total cars detected:   32036
Avg humans per image:  8.44
Avg cars per image:    19.90
Images with no humans: 585
Images with no cars:   82
Saved CSV:             /kaggle/working/detection_log.csv
=====
```

Top 10 images by human count:

image	humans	cars
0000066_01097_d_0000002.jpg	300	0
0000063_00500_d_0000001.jpg	171	0
0000063_12500_d_0000014.jpg	126	0
0000074_07850_d_0000015.jpg	123	2
0000063_08000_d_0000009.jpg	115	0
9999938_00000_d_0000213.jpg	109	26
0000087_00700_d_0000003.jpg	104	0
0000063_07000_d_0000008.jpg	103	0
0000074_09738_d_0000019.jpg	103	3
0000063_11000_d_0000012.jpg	101	4

Top 10 images by car count:

image	humans	cars
0000321_00000_d_0000015.jpg	19	149
9999979_00000_d_0000026.jpg	11	117
9999979_00000_d_0000045.jpg	11	116
0000127_00002_d_0000119.jpg	39	110
9999979_00000_d_0000023.jpg	11	110
0000127_00265_d_0000120.jpg	28	107
9999979_00000_d_0000047.jpg	21	96
0000278_02951_d_0000008.jpg	7	92
9999952_00000_d_0000233.jpg	0	86
9999979_00000_d_0000044.jpg	4	86

```
In [20]: # CELL 7-B: Visualize high-count detection examples from the test set

import matplotlib.pyplot as plt
import pandas as pd

# detect_and_count() from Cell 6 expects dictionaries, not sets
HUMAN_CLASSES = {0: 'pedestrian', 1: 'people'}
CAR_CLASSES = {3: 'car', 4: 'van'}

top_human_images = df_results.nlargest(3, 'humans')
top_car_images = df_results.nlargest(3, 'cars')

example_df = pd.concat([top_human_images, top_car_images]).drop_duplicates('image')
example_df = example_df.head(6)

print("Selected example images:")
print(example_df[['image', 'humans', 'cars']].to_string(index=False))
```

```

fig, axes = plt.subplots(2, 3, figsize=(22, 13))
fig.suptitle(
    "High-Count Detection Examples from Test Images",
    fontsize=16,
    fontweight='bold'
)

axes = axes.flatten()

for ax, (_, row) in zip(axes, example_df.iterrows()):
    img_path = test_img_dir / row['image']

    result_img, counts = detect_and_count(img_path, conf_threshold=0.25)

    ax.imshow(result_img)
    ax.set_title(
        f"{row['image']}\nHumans: {counts['humans']} | Cars: {counts['cars']}",
        fontsize=9
    )
    ax.axis('off')

for ax in axes[len(example_df):]:
    ax.axis('off')

plt.tight_layout()
plt.savefig('/kaggle/working/high_count_detection_examples.png', dpi=150, bbox_inches=
plt.show()

print("Saved: /kaggle/working/high_count_detection_examples.png")

```

Selected example images:

image	humans	cars
0000066_01097_d_0000002.jpg	300	0
0000063_00500_d_0000001.jpg	171	0
0000063_12500_d_0000014.jpg	126	0
0000321_00000_d_0000015.jpg	19	149
9999979_00000_d_0000026.jpg	11	117
9999979_00000_d_0000045.jpg	11	116

High-Count Detection Examples from Test Images



Saved: /kaggle/working/high_count_detection_examples.png

```
In [21]: # CELL 8: Official validation mAP evaluation using Ultralytics

from ultralytics import YOLO

model_best = YOLO(MODEL_PATH)

val_results = model_best.val(
    data='/kaggle/working/visdrone.yaml',
    split='val',
    imgsz=800,
    batch=16,
    device=0,
    plots=True,
    project='/kaggle/working/runs',
    name='visdrone_val_eval',
    verbose=True
)

print(f"\n🏆 OFFICIAL VALIDATION RESULTS")
print(f"{'='*40}")
print(f"mAP@50:      {val_results.box.map50:.4f}")
print(f"mAP@50-95:   {val_results.box.map:.4f}")
print(f"Precision:   {val_results.box.mp:.4f}")
print(f"Recall:      {val_results.box.mr:.4f}")
print(f"{'='*40}")

# Optional: per-class mAP@50-95
if hasattr(val_results.box, 'maps') and val_results.box.maps is not None:
    print("\nPer-class mAP@50-95:")
    names = val_results.names

    for class_id, class_map in enumerate(val_results.box.maps):
```

```
class_name = names.get(class_id, str(class_id)) if isinstance(names, dict)
print(f"{class_id:2d} | {class_name:16s} | {class_map:.4f}")
```

Ultralytics 8.4.51 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
 Model summary (fused): 73 layers, 11,129,454 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 122.1±64.5 MB/s, size: 120.8 KB)
val: Scanning /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/V
 isDrone2019-DET-val/labels... 548 images, 0 backgrounds, 0 corrupt: 100%

 - 548/548 610.1it/s 0.9s0.1s

WARNING ⚠️ **val:** Cache directory /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-val is not writable, cache not saved.

	Class	Images	Instances	Box(P	R	mAP50	mAP50
-95): 100%		35/35	3.0it/s	11.6s0.2s			
0.26	all	548	38759	0.556	0.441	0.437	
0.229	pedestrian	520	8844	0.568	0.501	0.502	
0.147	people	482	5125	0.594	0.361	0.382	
0.0872	bicycle	364	1287	0.319	0.231	0.192	0.
0.572	car	515	14064	0.756	0.812	0.822	
0.335	van	421	1975	0.562	0.474	0.477	
0.279	truck	266	750	0.561	0.404	0.412	
0.186	tricycle	337	1045	0.455	0.361	0.326	
0.101	awning-tricycle	220	532	0.347	0.216	0.164	
0.434	bus	131	251	0.789	0.558	0.586	
0.227	motor	485	4886	0.611	0.49	0.51	

Speed: 1.2ms preprocess, 7.3ms inference, 0.0ms loss, 4.5ms postprocess per image
 Results saved to /kaggle/working/runs/visdrone_val_eval-2

🏆 OFFICIAL VALIDATION RESULTS

=====

mAP@50: 0.4374
 mAP@50-95: 0.2598
 Precision: 0.5562
 Recall: 0.4407

=====

Per-class mAP@50-95:

0	pedestrian	0.2292
1	people	0.1473
2	bicycle	0.0872
3	car	0.5722
4	van	0.3352
5	truck	0.2789
6	tricycle	0.1860
7	awning-tricycle	0.1010
8	bus	0.4340
9	motor	0.2272

```

In [18]: # CELL 9: Create a smooth video-like sequence from VisDrone frames

import cv2
import re
import numpy as np
from pathlib import Path
from collections import defaultdict

# Automatically find the VisDrone train/images folder
root = Path('/kaggle/input')

candidate_dirs = []
for d in root.rglob('images'):
    imgs = list(d.glob('*.jpg')) + list(d.glob('*.JPG')) + list(d.glob('*.png'))
    if len(imgs) > 50 and 'VisDrone2019-DET-train' in str(d):
        candidate_dirs.append((d, len(imgs)))

if not candidate_dirs:
    raise FileNotFoundError("Could not find VisDrone2019-DET-train/images under /ka

train_img_dir = max(candidate_dirs, key=lambda x: x[1])[0]
print("Using image folder:", train_img_dir)

# VisDrone style: 0000001_00001_d_0000001.jpg
pattern = re.compile(r'^(\d+)_([0-9]+)_')

sequences = defaultdict(list)

for img_path in sorted(train_img_dir.glob('*')):
    if img_path.suffix.lower() not in ['.jpg', '.jpeg', '.png']:
        continue

    m = pattern.match(img_path.name)
    if m is None:
        continue

    seq_id = m.group(1)
    frame_num = int(m.group(2))
    sequences[seq_id].append((frame_num, img_path))

print(f"Found {len(sequences)} sequences")

# Split into continuous clips so distant frames do not look like jumps
MAX_FRAME_GAP = 3
MIN_CLIP_LEN = 40
clips = []

for seq_id, items in sequences.items():
    items = sorted(items, key=lambda x: x[0])

    current = [items[0]]

    for prev, curr in zip(items, items[1:]):
        prev_num = prev[0]
        curr_num = curr[0]

```

```

        if curr_num - prev_num <= MAX_FRAME_GAP:
            current.append(curr)
        else:
            if len(current) >= MIN_CLIP_LEN:
                clips.append((seq_id, current))
                current = [curr]

    if len(current) >= MIN_CLIP_LEN:
        clips.append((seq_id, current))

if not clips:
    raise RuntimeError("No continuous clips found. Try MAX_FRAME_GAP=5 or MIN_CLIP_

def smoothness_score(paths, sample_count=30):
    """
    Lower score means less visual jumping between consecutive frames.
    """
    sample_n = min(sample_count, len(paths) - 1)
    if sample_n <= 0:
        return float('inf')

    idxs = np.linspace(0, len(paths) - 2, sample_n).astype(int)
    diffs = []

    for i in idxs:
        a = cv2.imread(str(paths[i]))
        b = cv2.imread(str(paths[i + 1]))

        if a is None or b is None:
            continue

        a = cv2.cvtColor(cv2.resize(a, (160, 90)), cv2.COLOR_BGR2GRAY)
        b = cv2.cvtColor(cv2.resize(b, (160, 90)), cv2.COLOR_BGR2GRAY)

        diffs.append(np.mean(cv2.absdiff(a, b)))

    return np.mean(diffs) if diffs else float('inf')

scored = []

for seq_id, clip in clips:
    frame_nums = [x[0] for x in clip]
    paths = [x[1] for x in clip]

    avg_gap = np.mean(np.diff(frame_nums))
    jump_score = smoothness_score(paths)

    # Long clip + low frame gap + low visual jump = better video-like sequence
    score = len(paths) * 4 - avg_gap * 30 - jump_score

    scored.append({
        'seq_id': seq_id,
        'paths': paths,
        'length': len(paths),
        'avg_gap': avg_gap,

```

```

        'jump_score': jump_score,
        'score': score
    })

best = max(scored, key=lambda x: x['score'])
seq_frames = best['paths'][:150]

print("\nBest smooth clip:")
print("Sequence ID:", best['seq_id'])
print("Available frames:", best['length'])
print("Using frames:", len(seq_frames))
print("Average frame gap:", round(best['avg_gap'], 2))
print("Visual jump score:", round(best['jump_score'], 2))

first = cv2.imread(str(seq_frames[0]))
if first is None:
    raise RuntimeError("Could not read first frame")

h, w = first.shape[:2]

video_path = '/kaggle/working/input_sequence.mp4'
fps = 10

writer = cv2.VideoWriter(
    video_path,
    cv2.VideoWriter_fourcc(*'mp4v'),
    fps,
    (w, h)
)

for p in seq_frames:
    frame = cv2.imread(str(p))
    if frame is None:
        continue

    if frame.shape[:2] != (h, w):
        frame = cv2.resize(frame, (w, h))

    writer.write(frame)

writer.release()

print(f"\n✅ Input sequence video created: {video_path}")
print(f"Frames: {len(seq_frames)} | FPS: {fps} | Size: {w}x{h}")

```

Using image folder: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-train/images

Found 208 sequences

Best smooth clip:

Sequence ID: 9999955

Available frames: 434

Using frames: 150

Average frame gap: 0.0

Visual jump score: 53.99

✓ Input sequence video created: /kaggle/working/input_sequence.mp4
Frames: 150 | FPS: 10 | Size: 1400x788

In [23]: *# CELL 10: Detect humans/cars on every frame and show object identity with ByteTrac*

```
from ultralytics import YOLO
import cv2
import numpy as np
from pathlib import Path

model = YOLO(MODEL_PATH)

# VisDrone class IDs based on your yaml
CLASS_NAMES = {
    0: 'pedestrian',
    1: 'people',
    3: 'car'
}

HUMAN_CLASSES = {0, 1}
CAR_CLASSES = {3}
TARGET_CLASSES = HUMAN_CLASSES | CAR_CLASSES

video_path = '/kaggle/working/input_sequence.mp4'
output_path = '/kaggle/working/tracked_detection_output.mp4'

if not Path(video_path).exists():
    raise FileNotFoundError(f"Input video not found: {video_path}")

cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS)
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
cap.release()

if fps <= 0:
    fps = 10

BANNER_H = 55

writer = cv2.VideoWriter(
    output_path,
    cv2.VideoWriter_fourcc(*'mp4v'),
    fps,
    (w, h + BANNER_H))
```

```

)

frame_idx = 0

print("Running per-frame YOLO detection + ByteTrack IDs...")

for result in model.track(
    source=video_path,
    tracker='bytetrack.yaml',
    persist=True,
    stream=True,
    verbose=False,

    # Accuracy-focused settings
    imgsz=960,          # better for small VisDrone objects than 640
    conf=0.20,          # raise to 0.25/0.30 for fewer false positives
    iou=0.50,
    classes=list(TARGET_CLASSES)
):
    frame = result.orig_img.copy()
    frame_idx += 1

    humans_in_frame = 0
    cars_in_frame = 0

    if result.bboxes is not None and len(result.bboxes) > 0:
        boxes = result.bboxes.xyxy.cpu().numpy()
        cls_ids = result.bboxes.cls.cpu().numpy().astype(int)
        confs = result.bboxes.conf.cpu().numpy()

        if result.bboxes.id is not None:
            track_ids = result.bboxes.id.cpu().numpy().astype(int)
        else:
            track_ids = [-1] * len(boxes)

        for box, cls_id, conf, track_id in zip(boxes, cls_ids, confs, track_ids):
            if cls_id not in TARGET_CLASSES:
                continue

            x1, y1, x2, y2 = map(int, box)

            class_name = CLASS_NAMES.get(cls_id, str(cls_id))

            if cls_id in HUMAN_CLASSES:
                humans_in_frame += 1
                color = (0, 0, 255)          # red in BGR
                prefix = "HUMAN"
            elif cls_id in CAR_CLASSES:
                cars_in_frame += 1
                color = (255, 180, 60)      # blue/orange in BGR
                prefix = "CAR"
            else:
                continue

            if track_id >= 0:
                label = f"{prefix} #{track_id} {class_name} {conf:.2f}"

```



```

else:
    label = f"{prefix} {class_name} {conf:.2f}"

    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

    font_scale = 0.45
    thickness = 1
    (tw, th), _ = cv2.getTextSize(
        label,
        cv2.FONT_HERSHEY_SIMPLEX,
        font_scale,
        thickness
    )

    label_y = max(y1, th + 8)

    cv2.rectangle(
        frame,
        (x1, label_y - th - 6),
        (min(x1 + tw + 6, w - 1), label_y),
        color,
        -1
    )

    cv2.putText(
        frame,
        label,
        (x1 + 3, label_y - 4),
        cv2.FONT_HERSHEY_SIMPLEX,
        font_scale,
        (255, 255, 255),
        thickness
    )

# Banner shows CURRENT FRAME counts, not unique accumulated totals
banner = np.zeros((BANNER_H, w, 3), dtype=np.uint8)
banner[:] = (25, 25, 25)

cv2.putText(
    banner,
    f"Frame {frame_idx:03d}",
    (10, 36),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.75,
    (190, 190, 190),
    1
)

cv2.putText(
    banner,
    f"Humans detected now: {humans_in_frame}",
    (170, 36),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.75,
    (80, 120, 255),
    2

```

```

    )

    cv2.putText(
        banner,
        f"Cars detected now: {cars_in_frame}",
        (540, 36),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.75,
        (255, 190, 100),
        2
    )

    combined = np.vstack([banner, frame])
    writer.write(combined)

writer.release()

print(f"\n✅ Per-frame tracked detection video saved to: {output_path}")
print(f"Frames processed: {frame_idx}")
print("\nEach frame now shows:")
print("1. Every detected human/car with a box")
print("2. Class label")
print("3. Tracking ID")
print("4. Confidence score")
print("5. Current-frame human/car counts")

```

Running per-frame YOLO detection + ByteTrack IDs...

✅ Per-frame tracked detection video saved to: /kaggle/working/tracked_detection_output.mp4
Frames processed: 150

Each frame now shows:

1. Every detected human/car with a box
2. Class label
3. Tracking ID
4. Confidence score
5. Current-frame human/car counts

```

In [24]: import zipfile
        from pathlib import Path

        working_dir = Path('/kaggle/working')
        zip_path = working_dir / 'kaggle_working_files.zip'

        if zip_path.exists():
            zip_path.unlink()

        with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
            for file_path in working_dir.rglob('*'):
                if file_path.is_file() and file_path != zip_path:
                    zf.write(file_path, file_path.relative_to(working_dir))

        print(f"✅ Created zip file: {zip_path}")

```

✅ Created zip file: /kaggle/working/kaggle_working_files.zip